Redes Neuronales III

Optimización regularización y evaluación de modelos de Deep Learning

Estudiante: Ruby Dayana Cárdenas Gómez

Universidad: Universidad de Cundinamarca

CADI: Deep Learning Conceptos

Docente: Nidia Stella García Roa

Fecha: Octubre de 2026

1 Introducción

En esta entrega profundizo en el entrenamiento de redes neuronales con TensorFlow y Keras. El trabajo continúa las prácticas anteriores: primero implementé los componentes de una red con NumPy y después comparé implementaciones con Keras y PyTorch. Ahora centro el análisis en la selección de hiperparámetros, el control del sobreajuste y la evaluación con métricas que permiten examinar distintos tipos de error.

Desarrollé dos notebooks ejecutables en Google Colab. La primera práctica usa datos de diagnóstico de cáncer de mama y compara SGD con momentum, RMSprop y Adam. La segunda clasifica diez dígitos manuscritos. En ambos casos separé train, dev y test, apliqué preprocesamiento sin fuga de información y reservé test para la evaluación final.

Los experimentos alcanzaron 96.51 % de Accuracy en clasificación binaria y 98.52 % en clasificación multiclase. Estos valores se obtuvieron al ejecutar los notebooks completos con semilla 42. Las gráficas y matrices incluidas en este documento provienen de esa ejecución.

2 Objetivos

2.1 Objetivo general

Implementar, optimizar y evaluar redes neuronales con TensorFlow y Keras, aplicando separación de datos, preprocesamiento, optimizadores, regularización y métricas de clasificación.

2.2 Objetivos específicos

- Comparar SGD con momentum, RMSprop y Adam bajo una arquitectura común.
- Aplicar Dropout, regularización L2 y Early Stopping para controlar el sobreajuste.
- Separar los datos en train, dev y test mediante muestreo estratificado.
- Calcular Accuracy, Precision, Recall, F1 Score y matrices de confusión.
- Interpretar los resultados desde los conceptos de bias, variance y generalización.
- Documentar el código y los resultados en notebooks compatibles con Google Colab.

3 Fundamentos conceptuales

3.1 Hiperparámetros

Los hiperparámetros se definen antes o durante la configuración del entrenamiento; no son pesos aprendidos directamente por la red. Entre ellos se encuentran el learning rate, el tamaño del batch, el número de capas, la cantidad de neuronas, la función de activación, la intensidad de L2, la tasa de Dropout y la paciencia de Early Stopping. Su elección modifica la velocidad de convergencia, la capacidad del modelo y su desempeño fuera de train.

3.2 Conjuntos train dev test

Conjunto	Función	Uso en las prácticas
Train	Ajusta los pesos de la red	70 % de los registros
Dev	Compara configuraciones y controla la parada	15 % de los registros
Test	Mide la generalización final	15 % y una sola evaluación final

La separación se realizó antes del escalado y mantuvo la proporción de clases. El escalador de la primera práctica se ajustó solo con train; dev y test recibieron la transformación calculada con esas estadísticas. Este orden evita que información de evaluación influya en el entrenamiento.

3.3 Optimización

Optimizador	Mecanismo principal	Implicación
SGD con momentum	Acumula dirección de gradientes anteriores	Puede converger de forma estable con un learning rate adecuado
RMSprop	Escala cada actualización con un promedio de gradientes cuadrados	Se adapta a parámetros con magnitudes diferentes
Adam	Combina momentos de primer y segundo orden	Suele requerir poco ajuste inicial

3.4 Regularización

Dropout desactiva aleatoriamente una fracción de las neuronas durante train. L2 agrega a la pérdida una penalización proporcional al cuadrado de los pesos. Early Stopping detiene el entrenamiento cuando la pérdida de dev deja de mejorar y restaura los mejores pesos. Los tres mecanismos atacan el sobreajuste desde puntos distintos.

3.5 Bias variance y tradeoff

Un modelo con bias alto presenta subajuste: no representa bien el patrón y obtiene resultados bajos tanto en train como en dev. Un modelo con variance alta memoriza train y pierde rendimiento en dev. Aumentar capacidad puede reducir bias, pero también elevar variance. La regularización puede reducir variance a cambio de introducir una pequeña cantidad de bias.

3.6 Métricas de clasificación

Métrica	Pregunta que responde
Accuracy	Qué proporción de ejemplos se clasificó correctamente
Precision	De las predicciones de una clase cuántas fueron correctas
Recall	De los ejemplos reales de una clase cuántos fueron recuperados
F1 Score	Cuál es el equilibrio entre Precision y Recall
Matriz de confusión	Entre qué clases se concentran los aciertos y errores

4 Práctica de optimización y regularización

4.1 Datos y preprocesamiento

La primera práctica emplea Breast Cancer Wisconsin, con 569 registros, 30 variables numéricas y dos clases. El conjunto contiene 212 tumores malignos y 357 benignos. La partición estratificada produjo 398 registros de train, 85 de dev y 86 de test. StandardScaler centró y escaló las variables porque radios, texturas y medidas geométricas se expresan en rangos diferentes.

4.2 Arquitectura

Componente	Configuración
Entrada	30 variables estandarizadas
Capa oculta 1	32 neuronas ReLU con inicialización He
Capa oculta 2	16 neuronas ReLU con inicialización He
Salida	1 neurona sigmoide
Pérdida	Binary crossentropy
Batch	32 ejemplos

4.3 Resultados de optimización

Optimizador	Épocas	Mejor pérdida dev	Accuracy dev
SGD	55	0.0421	97.65 %
RMSprop	40	0.0422	97.65 %
Adam	59	0.0468	97.65 %

Los tres optimizadores alcanzaron la misma Accuracy en dev. SGD obtuvo la pérdida mínima más baja por una diferencia de 0.0001 frente a RMSprop, por lo que el criterio definido en el notebook lo seleccionó. RMSprop necesitó 40 épocas, 15 menos que SGD, y mostró la convergencia más rápida.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for nombre, historia in historias.items():
    axes[0].plot(historia["val_loss"], label=nombre)
    axes[1].plot(historia["val_accuracy"], label=nombre)
axes[0].set(title="Pérdida en dev", xlabel="Época", ylabel="Binary crossentropy")
axes[1].set(title="Exactitud en dev", xlabel="Época", ylabel="Accuracy")
for ax in axes:
    ax.legend()
plt.tight_layout()
plt.show()
```

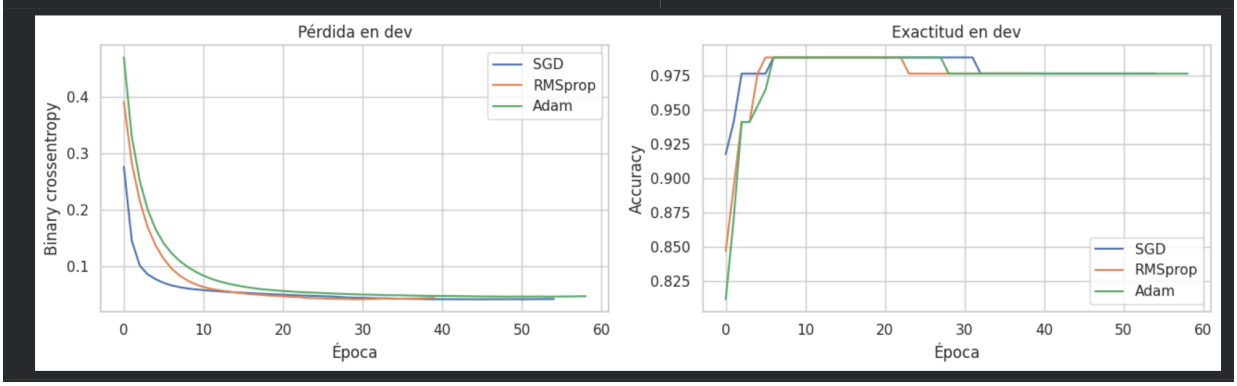


Figura 1 Comparación de pérdida y Accuracy en dev para los tres optimizadores

4.4 Resultados de regularización

El modelo regularizado mantuvo SGD e incorporó L2 de 0.0001, Dropout de 30 % y Early Stopping con paciencia de 12 épocas. El entrenamiento se detuvo después de 34 épocas. El modelo base y el regularizado alcanzaron las mismas métricas en test; por tanto, la regularización no mejoró el porcentaje final en esta partición, pero conservó la capacidad de generalización con un entrenamiento más corto.

Modelo	Accuracy	Precision	Recall	F1
Base SGD	96.51 %	98.11 %	96.30 %	97.20 %
L2 Dropout Early Stopping	96.51 %	98.11 %	96.30 %	97.20 %

```
plt.figure(figsize=(8, 4))
plt.plot(historia_regularizada.history["loss"], label="Train")
plt.plot(historia_regularizada.history["val_loss"], label="Dev")
plt.title("Modelo regularizado")
plt.xlabel("Época")
plt.ylabel("Pérdida")
plt.legend()
plt.tight_layout()
plt.show()
```

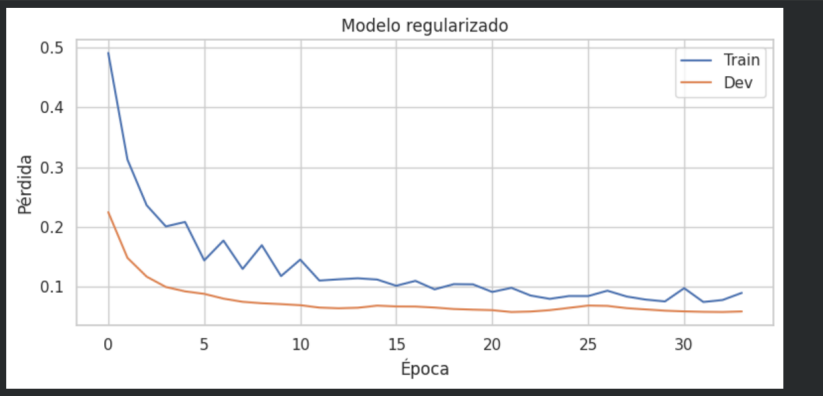


Figura 2 Evolución de la pérdida del modelo regularizado

4.5 Matriz de confusión

El modelo clasificó correctamente 31 tumores malignos y 52 benignos. Hubo un maligno clasificado como benigno y dos benignos clasificados como malignos. En este conjunto, la clase positiva usada por las métricas es benigno; por eso Precision y Recall deben leerse junto con la matriz.

```
cm = confusion_matrix(y_test, pred_reg)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["Maligno", "Benigno"], yticklabels=["Maligno", "Benigno"])
plt.title("Matriz de confusión del modelo regularizado")
plt.xlabel("Predicción")
plt.ylabel("Valor real")
plt.tight_layout()
plt.show()
```

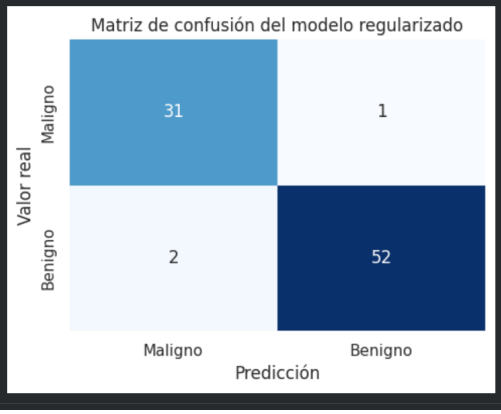


Figura 3 Matriz de confusión de la clasificación binaria

5 Práctica de evaluación multiclase

5.1 Datos y preprocesamiento

La segunda práctica utiliza el conjunto Digits de scikit-learn. Contiene 1 797 imágenes de 8 por 8 píxeles y diez clases. Las intensidades originales, entre 0 y 16, se dividieron por 16 para llevarlas al intervalo de 0 a 1. Las etiquetas se transformaron a one hot encoding. La partición dejó 1 257 imágenes en train, 270 en dev y 270 en test.



Figura 4 Ejemplo de cada dígito del conjunto de datos

5.2 Arquitectura y entrenamiento

Componente	Configuración
Entrada	64 intensidades normalizadas
Capa oculta 1	128 ReLU L2 0.0001 Dropout 25 %
Capa oculta 2	64 ReLU L2 0.0001 Dropout 20 %
Salida	10 neuronas softmax
Optimizador	Adam con learning rate 0.001
Control	Early Stopping y ReduceLROnPlateau

El entrenamiento ejecutó 35 épocas antes de que Early Stopping restaurara los pesos asociados con la mejor pérdida de dev. ReduceLROnPlateau disminuyó el learning rate cuando esa pérdida dejó de mejorar durante cuatro épocas.

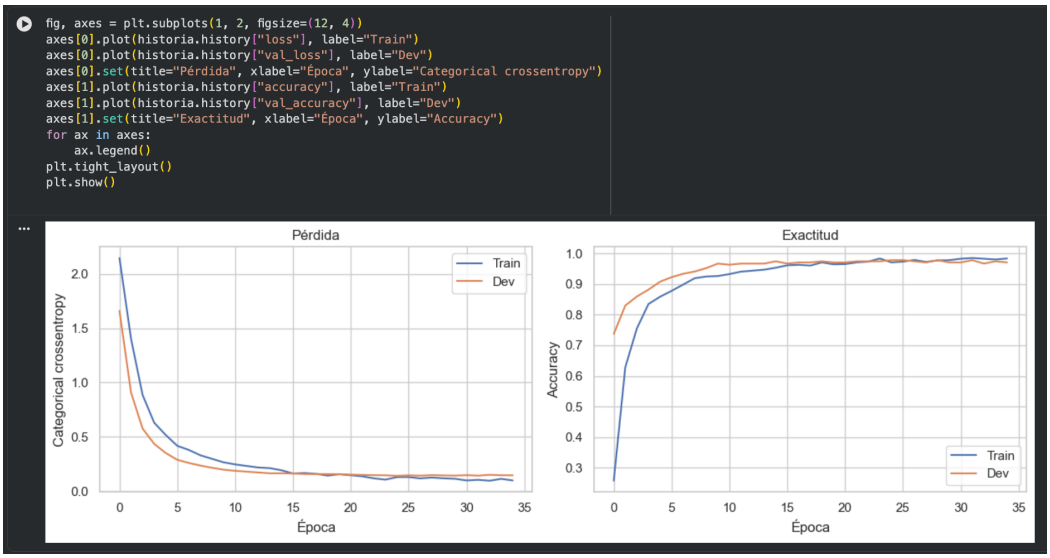


Figura 5 Curvas de entrenamiento y validación del modelo multiclase

5.3 Métricas sobre test

Accuracy	Precision macro	Recall macro	F1 macro	Errores
98.52 %	98.54 %	98.50 %	98.52 %	4 de 270

Las cuatro métricas quedaron alrededor de 98.5 %. Su cercanía es coherente con la distribución equilibrada de las clases. El promedio macro evita que una clase con más ejemplos domine el resultado.

5.4 Matriz de confusión y análisis de errores

El modelo cometió cuatro errores: confundió dos ochos con unos, un uno con un ocho y un seis con un ocho. La confusión principal se relaciona con trazos de forma similar en imágenes de baja resolución. Las demás clases se clasificaron sin errores en esta partición.

8 Matriz de confusión

La diagonal contiene los aciertos. Los valores fuera de ella muestran qué dígitos se confunden y permiten identificar errores que Accuracy no explica.

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.title("Matriz de confusión en test")
plt.xlabel("Dígito predicho")
plt.ylabel("Dígito real")
plt.tight_layout()
plt.show()
```

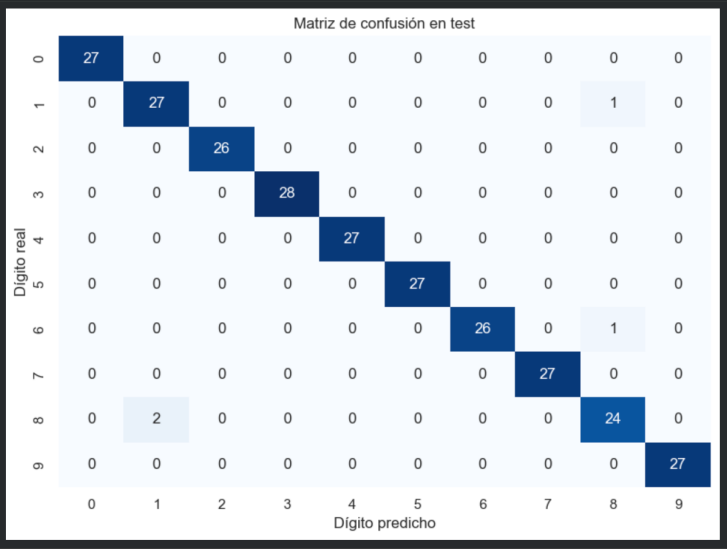


Figura 6 Matriz de confusión del modelo multiclase

```
errores = np.where(y_test != y_pred)[0]
cantidad = min(8, len(errores))
fig, axes = plt.subplots(1, max(cantidad, 1), figsize=(2.2 * max(cantidad, 1), 2.8))
axes = np.atleast_1d(axes)
for ax, indice_local in zip(axes, errores[:cantidad]):
    ax.imshow(X_test[indice_local].reshape(8, 8), cmap="gray_r")
    ax.set_title(f"Real {y_test[indice_local]} / Pred {y_pred[indice_local]}")
    ax.axis("off")
plt.suptitle("Ejemplos clasificados incorrectamente")
plt.tight_layout(rect=[0, 0, 1, 0.90])
plt.show()
```

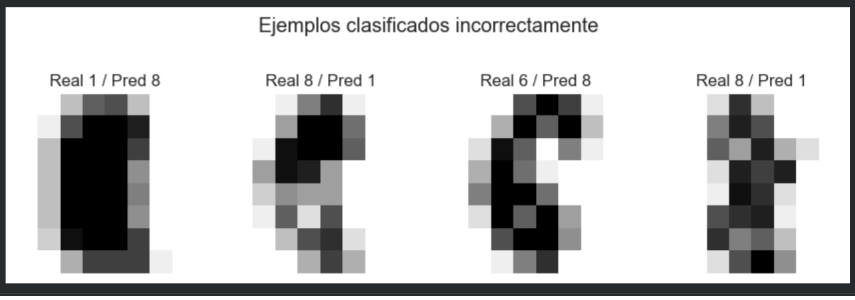


Figura 7 Imágenes clasificadas incorrectamente

6 Discusión de resultados

6.1 Optimización

La comparación no mostró un ganador amplio en Accuracy: los tres optimizadores alcanzaron 97.65 % en dev. La diferencia apareció en la velocidad y en la pérdida mínima. RMSprop terminó antes, mientras SGD produjo la pérdida dev más baja por un margen pequeño. Este resultado confirma que evaluar un optimizador solo por Accuracy puede ocultar diferencias de convergencia.

6.2 Regularización y tradeoff

La regularización conservó las métricas del modelo base y redujo el número de épocas de 55 a 34. No existe evidencia en este experimento de una mejora de Accuracy, porque ambos modelos acertaron los mismos 83 casos de 86. El beneficio observable fue mantener el desempeño mientras Dropout, L2 y Early Stopping limitaron la dependencia de un ajuste prolongado.

6.3 Evaluación multiclase

La red multiclase acertó 266 de 270 imágenes. Precision, Recall y F1 macro confirmaron que el resultado no dependió de una sola clase. La matriz identificó una concentración de errores entre 1 y 8. Esa información orienta una mejora futura más concreta que aumentar épocas: revisar esos trazos, aplicar aumentación de datos o probar una red convolucional que conserve la estructura espacial.

6.4 Limitaciones

Los resultados corresponden a una partición aleatoria con semilla 42. Otra semilla puede cambiar ligeramente las métricas. Tampoco se realizó una búsqueda exhaustiva de hiperparámetros; se compararon configuraciones controladas para estudiar sus efectos. En Digits se usó una red densa, aunque una red convolucional sería una extensión natural para imágenes de mayor resolución.

7 Buenas prácticas aplicadas

- Uso de una semilla común para mejorar la reproducibilidad de la comparación.
- Partición estratificada train dev test antes del preprocesamiento.
- Ajuste de StandardScaler únicamente con train.
- Arquitectura constante durante la comparación de optimizadores.
- Selección de configuración con dev y evaluación final separada sobre test.
- Funciones reutilizables para construir modelos y calcular métricas.
- Comentarios, títulos y explicaciones técnicas dentro de cada notebook.
- Visualización de curvas de aprendizaje y matrices de confusión.

8 Conclusiones

Los experimentos muestran que la evaluación de una red neuronal requiere separar el proceso de selección del resultado final. Dev permitió comparar optimizadores y controlar la parada sin consultar test. Esta separación redujo el riesgo de adaptar decisiones al conjunto usado para informar el resultado.

En la clasificación binaria, SGD, RMSprop y Adam alcanzaron la misma Accuracy en dev. SGD obtuvo la menor pérdida, mientras RMSprop convergió en menos épocas. El modelo regularizado logró 96.51 % de Accuracy y un F1 de 97.20 % en test. En la clasificación de dígitos, el modelo alcanzó 98.52 % de Accuracy y cometió cuatro errores.

Accuracy ofreció una visión general, pero Precision, Recall, F1 y las matrices de confusión permitieron examinar el tipo de error. Esta combinación de métricas, junto con un preprocesamiento correcto y regularización explícita, produjo una evaluación más completa y reproducible.

9 Repositorio y ejecución en Google Colab

URL: https://github.com/RubyDayana/redes-neuronales-iii.git

Anexo de código fuente

A. Construcción del modelo binario

```python
def crear_modelo(optimizador, l2_strength=0.0, dropout_rate=0.0):
    penalizacion = regularizers.l2(l2_strength) if l2_strength > 0 else None
    modelo = Sequential([
        Input(shape=(X_train_s.shape[1],)),
        Dense(32, activation="relu", kernel_initializer="he_normal", kernel_regularizer=penalizacion),
        Dropout(dropout_rate),
        Dense(16, activation="relu", kernel_initializer="he_normal", kernel_regularizer=penalizacion),
        Dropout(dropout_rate),
        Dense(1, activation="sigmoid"),
    ])
    modelo.compile(
        optimizer=optimizador,
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    return modelo
```

B. Separación y escalado

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)
X_dev, X_test, y_dev, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp
)

particiones = pd.DataFrame({
    "Partición": ["Train", "Dev", "Test"],
    "Registros": [len(X_train), len(X_dev), len(X_test)],
    "Porcentaje": [len(X_train), len(X_dev), len(X_test)],
})
particiones["Porcentaje"] = (particiones["Porcentaje"] / len(X) * 100).round(2)
display(particiones)

for nombre, etiquetas in [("Train", y_train), ("Dev", y_dev), ("Test", y_test)]:
    print(nombre, etiquetas.value_counts(normalize=True).sort_index().round(3).to_dict())
```

	Partición	Registros	Porcentaje
0	Train	398	69.95
1	Dev	85	14.94
2	Test	86	15.11

```
Train {0: 0.372, 1: 0.628}
Dev {0: 0.376, 1: 0.624}
Test {0: 0.372, 1: 0.628}
```

C. Modelo multiclase

```python
tf.keras.backend.clear_session()
tf.keras.utils.set_random_seed(SEED)

modelo = Sequential([
    Input(shape=(64,)),
    Dense(128, activation="relu", kernel_initializer="he_normal",
          kernel_regularizer=regularizers.l2(1e-4)),
    Dropout(0.25),
    Dense(64, activation="relu", kernel_initializer="he_normal",
          kernel_regularizer=regularizers.l2(1e-4)),
    Dropout(0.20),
    Dense(10, activation="softmax"),
])

modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
modelo.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	8,320
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 10)	650

Total params: 17,226 (67.29 KB)
Trainable params: 17,226 (67.29 KB)
Non-trainable params: 0 (0.00 B)

El código completo, las tablas por clase, las gráficas y los resultados de ejecución se encuentran en los dos notebooks entregados junto con este documento.